

VALAY SAITWADEKAR

Los Angeles, CA 92831 | ReachValay@Gmail.com | +1 (657) 319-8582 | LinkedIn.com/in/SaitwadekarValay/ | Valay17.GitHub.io/Portfolio/

EDUCATION

California State University Fullerton, United States of America
Master of Science in Computer Science

May 2026
CGPA 3.95/4

Shree L. R. Tiwari College of Engineering, Mumbai University, India
Bachelor of Engineering in Computer Engineering with
Honors in Cyber Security with *Distinction*

May 2024
CGPA 9.11/10
(3.65/4)

PROFESSIONAL EXPERIENCE

Filtroll International (India) Pvt. Ltd., Mumbai, India

December 2022 – June 2023

Intern – Software Engineer

- Implemented inventory management, accounting, and CRM modules using Python and Flask within the ERP system to help the business manage inventory levels and track financial transactions more efficiently, along with custom reports, dashboards, and analytics to provide business insights, reducing manual work by 50% and improving inventory management accuracy by 85%.
- Formed ERP system requirements and specifications document based on meetings with key business stakeholders to understand their needs and planned development of new capabilities aligned with business growth.
- Designed a relational database schema in MySQL to organize business data for reports and analysis with stored procedures and database triggers to enforce business logic and data integrity.
- Created user guides and admin manuals to support onboarding new users and documentation for users covering system navigation and module functions and conducted training to help employees adopt the new ERP tools.

PROJECTS

NanoParseTick – C++23 FIX 4.4 Signal Engine with Single-Instrument L2 Order Book and SBE-UDP Multicast

- Built a market-data engine that ingests the full IEX DEEP feed of 117.8M FIX messages at wire rate, filters to the subscribed instrument, tracks a live L2 order book, generates signals, and broadcasts book updates over SBE-UDP multicast off the hot thread. Sustained 1.21M messages/second with zero NIC-to-engine drops through traffic bursts via non-blocking busy-poll socket reads, and benchmarked a peak of 4.4M messages/second at 226 ns/message with a 6.8 us median order round-trip.
- Tuned the kernel and CPU to cut worst-case I/O-to-hot tail latency from 630 us to 17 us while holding p50 and p99 flat at 120 ns and 220 ns, targeting the tail rather than the median. Validated the full pipeline through a 3x load burst, sequence-gap injection, and repeated disconnect-reconnect cycles recovering in under 0.30 s, sustaining zero ring drops in every phase while p99 rose only to 250 ns at peak.
- Cut FIX parse latency to 191 ns/message over 1M+ messages with custom fixed-point numeric parsers, a decimal parser at 4.65 ns running 14x faster than strtod and an integer parser at 7.56 ns running 2x faster than atoi, floating-point-free on the hot path. Layered a complete FIX session engine over it for logon, heartbeat, sequence tracking, and gap recovery, requesting a resend from the server to pull missed messages.
- Bridged four pinned threads over custom wait-free SPSC ring buffers at a 140 ns round-trip with a zero-copy produce/consume path for 4 KB frames, running an allocation-free, lock-free, cache-aligned, false-sharing-free hot path. Replaced std::map with a cache-conscious flat sorted array to drop order-book update p999 from 50 ns to 20 ns, and discarded an AVX2 SoA book after it measured 64.4 ns against the scalar 41.7 ns.

DupliCron – Metadata-Based Incremental File Synchronization and Backup

- Built a multi-threaded C++20 tool for Windows and Linux that performs incremental backups from multiple source directories by detecting file changes using file size and modification time, with a binary metadata cache to prevent redundant copying across runs.
- Parallelized scanning, hashing, and copying phases using a custom thread pool to efficiently manage concurrent tasks, maximizing resource usage and reducing synchronization time through overlapping operations. Tested with over a million files, maintaining a low memory footprint of just a few megabytes.
- Designed a disk-aware per-source copy mechanism using thread-safe queues with serialized and parallel modes for peak throughput. Ensured data integrity by updating metadata only after verified copies and enabling automatic retries on failure.
- Implemented structured logging, UTF-8 support for multilingual file and folder names, and configured the system to be fully driven by a user-editable file supporting flexible sync modes, source exclusions, and stale file cleanup.

Open Source Contribution: Implemented and merged a conditional delete command (DELEX) into Apache Kvrocks, a distributed C++ key-value store.

TECHNICAL SKILLS

- Programming Languages:** C, C++ (keeping pace with C++26), Java, Kotlin, Python, SQL, R, Shell Scripting, HTML, CSS, JavaScript, TypeScript
- C++ Core:** Memory Model, RAII, Copy/Move Semantics, Smart Pointers, Perfect Forwarding, Value Categories, STL Internals, Custom Allocators, VTable Internals, Templates & Metaprogramming, SFINAE, Concepts, Type Traits, constexpr/constexpr, Object Lifetime, Storage Duration, EBO
- Concurrency:** Lock-Free/Wait-Free Programming, SPSC/MPMC Queues, Ring Buffers, Atomics, Memory Ordering, False Sharing, Memory Barriers, Hazard Pointers, RCU, Thread Pinning/Affinity, TLS, Thread Pools, Futex, Condition Variables, Double Buffering, Concurrent Data Structures
- Low Latency & Performance:** Hot Path Optimization, Cache Line Alignment, Branchless Coding, Zero-Copy Design, Fixed-Point Arithmetic, Vectorization, AVX2/SIMD Intrinsics, Loop Unrolling, Allocation-Free Hot Path, Deterministic/Bounded Latency, RVO/NRVO, LTO, GCC/Clang Function Attributes, Lock-Free/Cache-Conscious Data Structures, Inlining
- CPU & Microarchitecture:** x86, Cache Hierarchy, Out-of-Order Execution, Branch Prediction, CPU Pipelining, NUMA Topology, Virtual Memory, SIMD, Prefetching, Instruction-Level Parallelism, Superscalar Execution, Hyperthreading, Write/Store Buffers, Kernel/User Space, Clock Gating
- Hardware & System Tuning:** CPU/IRQ Affinity, Core Isolation, Huge Pages, NUMA Tuning, DMA, Memory-Mapped I/O, Memory Locking, C/P-State Management, Busy-Polling, Kernel Bypass, Cache Warming
- Debugging & Profiling:** perf, RDTSCP Cycle Timing, Flamegraphs, Callgrind/KCachegrind, Google Benchmark, HdrHistogram, Valgrind, ASan/UBSan/TSan, gdb, strace
- Build & Tools:** CMake, Make, Bazel, GTest, Git/GitHub, CI/CD, Test Driven Development, Unit & Integration Testing
- Distributed Systems:** Kafka, Event-driven Architecture, Real-time Systems, Fault Tolerance, AWS, Docker, Kubernetes, Prometheus, Grafana

RELEVANT COURSEWORK

- | | | |
|--------------------------------|--|----------------------------------|
| • Operating Systems Design | • Theory of Computer Science | • Microprocessor |
| • Advanced Algorithm Analysis | • Digital Logic & Computer Architecture | • Distributed Computing |
| • Advanced Computer Networking | • System Programming & Compiler Construction | • Software Design & Architecture |